



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251182
Nama Lengkap	Michael Dylan
Minggu ke / Materi	14 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU 1 (40%)

14.3.1 Pengenalan dan Mendefinisikan Set

Set adalah salah satu tipe data di Python yang digunakan untuk menyimpan kumpulan data unik atau tidak boleh ada yang sama. Isi dalam Set disebut anggota (member). Anggota pada Set harus bersifat immutable atau tidak bisa diubah, seperti integer, float, string, dan tuple, sehingga list dan dictionary tidak bisa dimasukkan ke dalam Set karena bersifat mutable. Walaupun begitu, Set sendiri bersifat mutable, jadi isinya masih bisa ditambah atau dikurangi. Karena Set bisa diubah, maka Set juga tidak bisa dimasukkan ke dalam Set lainnya. Dalam Python, Set bisa dibuat menggunakan tanda kurung kurawal {} atau menggunakan fungsi set(). Seperti di bawah ini:

```
Bilangan_genap = {2, 4, 6, 7, 8}      # dengan menggunakan {}  
pernah_ke_mars = set('Dylan')        # dengan menggunakan fungsi set()
```

Mendefinisikan Set kosong tidak dapat menggunakan notasi {}, tetapi menggunakan fungsi set() seperti contoh berikut:

```
pernah_ke_mars = set()                # menghasilkan set kosong  
Bilangan_genap = {}                  # ini akan menghasilkan dictionary kosong
```

14.3.2 Pengaksesan Set

Set tidak memiliki indeks, karena itu tidak dapat mengakses anggota-anggota dari sebuah Set secara langsung menggunakan indeks. Contoh di bawah ini:

```
nim = {'71200120', '71200195', '71200214'}  
jumlah_nim = len(nim)  
print(jumlah_nim) # akan menghasilkan output 3  
for n in nim:     # tampilkan isi set satu-persatu  
    print(n)
```

Output program di atas seperti di bawah ini:

```
3  
71200214  
71200195  
71200120
```

Pada tipe data Set, urutan output bisa berbeda dengan urutan saat data ditulis karena Set tidak memiliki indeks maupun urutan posisi anggota. Jadi, posisi data di dalam Set tidak dianggap penting. Selain itu, Set bersifat mutable, artinya isi datanya bisa ditambah atau dikurangi. Untuk menambahkan anggota baru ke dalam Set, dapat menggunakan fungsi `add()`:

```

tes2.py > ...
1 plat_nomor = set()           # definisikan sebuah set kosong
2 plat_nomor.add('AB 1890 XA') # tambahkan plat nomor 'AB 1890 XA'
3 plat_nomor.add('AD 6810 MT') # tambahkan plat nomor 'AD 6810 MT'
4 print(len(plat_nomor))       # jumlah anggota di dalam Set
5 plat_nomor.add('AB 1890 XA') # tambahkan plat yang sama sekali lagi
6 for plat in plat_nomor:      # tampilkan semua plat nomor
7     print(plat)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/
2
AB 1890 XA
AD 6810 MT

```

Gambar 14.1: menggunakan fungsi `add()` beserta outputnya. Sumber Python Vscode

Set memiliki fitur untuk mengecek duplikasi data secara otomatis. Jadi, saat menggunakan fungsi `add()`, Python akan memeriksa apakah data tersebut sudah ada di dalam Set atau belum. Jika belum ada, data akan ditambahkan, tetapi jika nilainya sama dengan anggota yang sudah ada, maka data tidak akan ditambahkan lagi. Karena pengecekan ini sudah dilakukan otomatis oleh fungsi `add()`, pengguna tidak perlu mengeceknya secara manual. Selain menambah data, Set juga dapat menghapus anggota menggunakan beberapa fungsi seperti `discard()`, `remove()`, `pop()`, dan `clear()`, yang masing-masing memiliki cara kerja berbeda seperti di Gambar 14.2.

<code>discard()</code>	<code>remove()</code>	<code>pop()</code>	<code>clear()</code>
Menghapus satu elemen yang disebutkan	Menghapus satu elemen yang disebutkan	Mengambil salah satu dan menghapusnya dari set (tidak tentu)	Menghapus seluruh elemen di dalam set
Tidak ada error	Muncul error jika elemen yang dihapus tidak ada	Error jika set kosong	Tidak ada error

Gambar 14.2: Cara kerja 4 fungsi `discard()`, `remove()`, `pop()`, `clear()`. Sumber Modul PrakAlpro

Berikut adalah contoh program yang mendemonstrasikan penghapusan anggota dari sebuah Set dengan menggunakan keempat fungsi seperti di Gambar 14.2:

```
tes2.py > ...
1  bilangan_prima = {13, 23, 7, 29, 11, 5}
2  print(bilangan_prima)
3  bilangan_prima.remove(5)      # hapus 5 dari set tersebut
4  print(bilangan_prima)
5  bilangan_prima.discard(97)    # hapus 97 (tidak ada)
6  print(bilangan_prima)
7  bilangan = bilangan_prima.pop() # ambil dan hapus salah satu
8  print(bilangan)
9  print(bilangan_prima)
10 bilangan_prima.clear()        # kosongkan set
11 print(bilangan_prima)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-
{29, 23, 5, 7, 11, 13}
{29, 23, 7, 11, 13}
{29, 23, 7, 11, 13}
29
{23, 7, 11, 13}
set()
```

Gambar 14.3: Penggunaan 4 fungsi discard(), remove(), pop(), clear(). Sumber Python Vscode

Fungsi “discard()” digunakan untuk menghapus anggota Set tanpa menimbulkan error jika data yang ingin dihapus ternyata tidak ada. Sedangkan fungsi “pop()” akan mengambil dan menghapus salah satu anggota Set secara acak. Fungsi ini berguna jika ingin memproses isi Set satu per satu tanpa memperhatikan urutan data. Selain itu, nilai anggota dalam Set tidak bisa diubah secara langsung. Jika ingin mengganti suatu nilai, caranya adalah dengan menghapus anggota lama terlebih dahulu, lalu menambahkan anggota baru yang diinginkan. Contoh programnya seperti di bawah ini:

```
tes2.py > ...
1  ikan = set(['koi', 'koki', 'kembung', 'salmon']) # Buat Set dari List
2  print(ikan)
3  ikan.remove('koi') # ganti koi menjadi teri
4  ikan.add('teri')
5  print(ikan)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/py
{'koki', 'kembung', 'salmon', 'koi'}
{'koki', 'kembung', 'salmon', 'teri'}
```

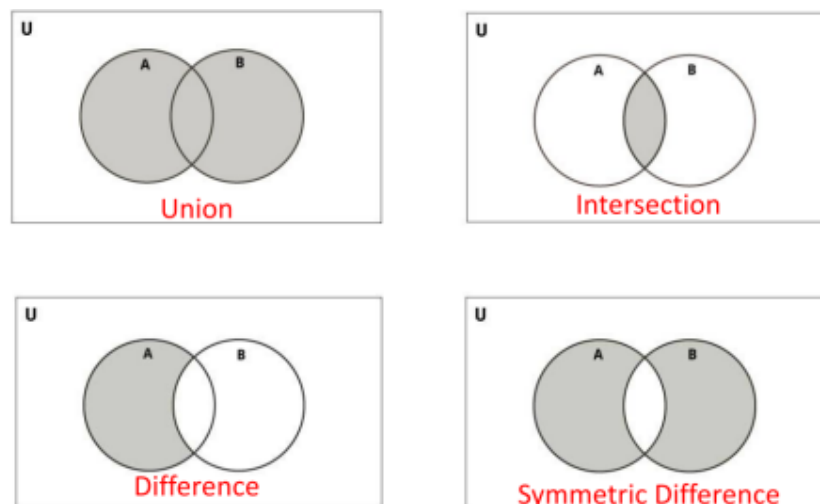
Gambar 14.4: Operasi Dasar Set Python (Tambah & Hapus Elemen). Sumber Python Vscode

Pada Set, nilai anggota tidak bisa diubah secara langsung. Jika ingin mengganti suatu data, caranya adalah dengan menghapus nilai lama terlebih dahulu menggunakan “remove()”, lalu

menambahkan nilai baru memakai “add()”. Contohnya pada Set “ikan”, data “koi” dihapus kemudian diganti menjadi “teri”. Selain itu, Set tidak memiliki urutan atau posisi data, sehingga urutan output bisa berubah setelah ada proses penambahan atau penghapusan anggota. Yang terpenting adalah data lama sudah terganti dengan data baru, bukan posisi urutannya.

14.3.3 Operasi-Operasi pada Set

Pada tipe data Set, terdapat beberapa operasi himpunan yang bisa digunakan untuk mengolah data. Operasi Union digunakan untuk menggabungkan dua Set menjadi satu dan dapat memakai operator `|` atau fungsi `union()`. Operasi Intersection digunakan untuk mencari anggota yang sama dari dua Set dengan operator `&` atau fungsi `intersection()`. Operasi Difference digunakan untuk mencari selisih anggota antara dua Set menggunakan operator `-` atau fungsi `difference()`. Sedangkan Symmetric Difference digunakan untuk menghasilkan anggota dari dua Set tanpa mengambil bagian yang sama atau irisannya, yang dapat dilakukan dengan operator `^` atau fungsi `symmetric_difference()`. Ilustrasi keempat fungsi operasi bisa dilihat pada Gambar 14.5.



Gambar 14.5: Ilustrasi keempat fungsi operasi. Sumber Modul PrakAlpro

OPERATOR UNION

```
tes2.py > ...
1  merek_hp = {'Samsung', 'Apple', 'Xiaomi', 'Sony'}
2  merek_ac = {'LG', 'Samsung', 'Panasonic', 'Daikin', 'Sony'}
3  gabungan = merek_hp | merek_ac # union dari merek_hp dan merek_ac
4  print(gabungan)                # bisa juga menggunakan gabungan = merek_hp.union(merek_ac)
```

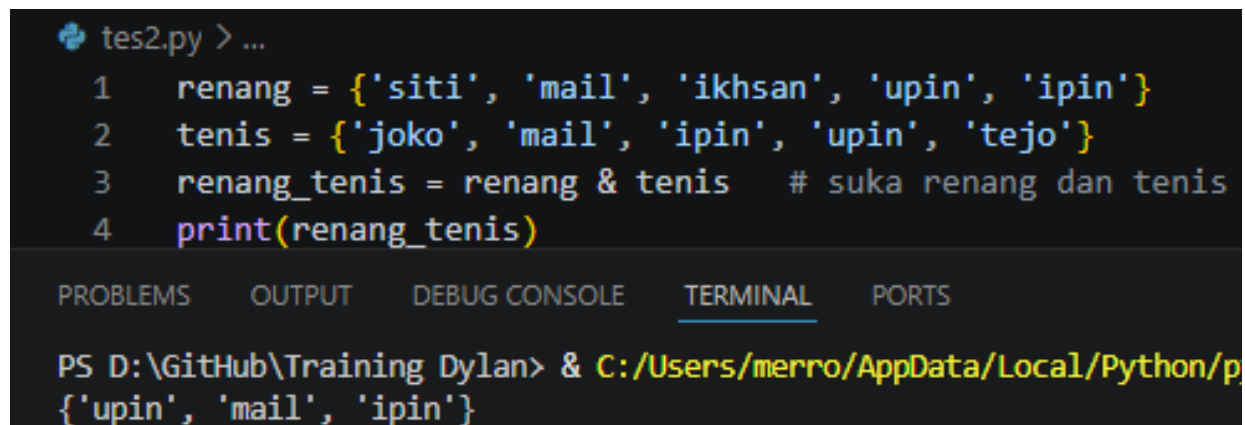
PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/python.exe "d:/GitHub/{'Xiaomi', 'Sony', 'LG', 'Samsung', 'Apple', 'Panasonic', 'Daikin'}
```

Gambar 14.6: Program menggunakan operator Union. Sumber Python Vscode

Output dari program tersebut menunjukkan bahwa anggota dari Set merek_hp digabungkan dengan anggota dari Set merek_ac sehingga menghasilkan Set baru bernama gabungan. Pada proses penggabungan tersebut terdapat data yang sama, yaitu “Samsung” dan “Sony”s. Karena Set hanya menyimpan data unik, maka anggota yang sama tersebut hanya akan muncul satu kali di dalam Set gabungan.

OPERATOR INTERSECTION



```
tes2.py > ...
1 renang = {'siti', 'mail', 'ikhshan', 'upin', 'ipin'}
2 tenis = {'joko', 'mail', 'ipin', 'upin', 'tejo'}
3 renang_tenis = renang & tenis # suka renang dan tenis
4 print(reenang_tenis)
```

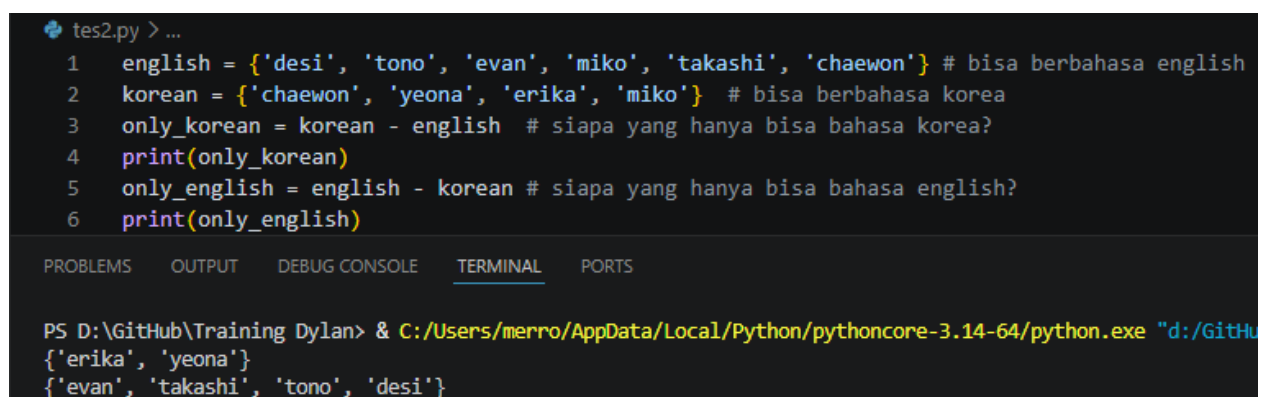
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/p
{'upin', 'mail', 'ipin'}
```

Gambar 14.7: Program menggunakan Operator Intersection. Sumber Python Vscode

Program tersebut menghasilkan output dari operasi intersection antara Set renang dan tenis. Operasi intersection digunakan untuk mencari anggota yang terdapat pada kedua Set sekaligus. Jadi, hasilnya adalah data yang sama-sama ada di dalam Set renang dan Set tenis.

OPERATOR DIFFERENCE



```
tes2.py > ...
1 english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'} # bisa berbahasa english
2 korean = {'chaewon', 'yeona', 'erika', 'miko'} # bisa berbahasa korea
3 only_korean = korean - english # siapa yang hanya bisa bahasa korea?
4 print(only_korean)
5 only_english = english - korean # siapa yang hanya bisa bahasa english?
6 print(only_english)
```

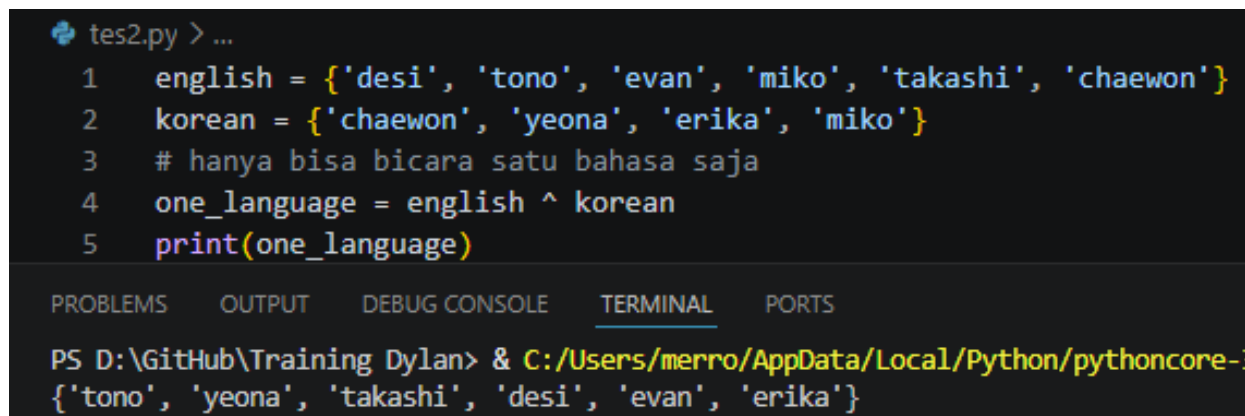
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/python.exe "d:/GitHu
{'erika', 'yeona'}
{'evan', 'takashi', 'tono', 'desi'}
```

Gambar 14.8: Program menggunakan Operator Difference. Sumber Python Vscode

Operator difference digunakan untuk menghasilkan Set yang berisi anggota hasil selisih dari dua Set yang dibandingkan. Pada contoh tersebut, operasi ini dipakai untuk mengetahui anggota yang hanya bisa berbahasa Korea dengan mencari selisih antara Set korean dan Set english. Sebaliknya, jika ingin mengetahui anggota yang hanya bisa berbahasa English, maka yang dicari adalah selisih antara Set english dan Set korean.

OPERATOR SYMMETRIC DIFFERENCE



```
tes2.py > ...
1  english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
2  korean = {'chaewon', 'yeona', 'erika', 'miko'}
3  # hanya bisa bicara satu bahasa saja
4  one_language = english ^ korean
5  print(one_language)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-:
{'tono', 'yeona', 'takashi', 'desi', 'evan', 'erika'}
```

Gambar 14.9: Program menggunakan Operator Symmetric Difference. Sumber Python Vscod Operator symmetric difference digunakan untuk menghasilkan Set baru yang merupakan gabungan dari dua Set tetapi tidak mengambil anggota yang sama atau irisannya. Pada contoh tersebut, operasi ini digunakan untuk mengetahui siapa saja yang hanya bisa berbicara dalam satu bahasa. Hasil yang diperoleh adalah anggota yang hanya ada di salah satu Set, bukan yang ada di keduanya. Operasi ini juga bisa dibuat dengan menggabungkan union lalu dikurangi hasil intersection Seperti di bawah ini:

```
one_languange = english.union(korean) - english.intersection(korean)
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github = https://github.com/MichaelDylan-ti/71251182_michaeldylan/tree/main

LATIHAN 14.1

Dari contoh kasus kategori kasus di Play Store, tambahkan kemampuan-kemampuan berikut ini:

- Tampilkan nama-nama aplikasi yang hanya muncul di satu kategori saja.
- Untuk input $n > 2$, tampilkan nama-nama aplikasi yang muncul tepat di dua kategori sekaligus

JAWABAN:

```
1  n = int(input('Masukkan jumlah kategori: '))
2  data_aplikasi = {}
3
4  for i in range(n):
5      nama_kategori = input('Masukkan nama kategori: ')
6      print('Masukkan 5 nama aplikasi di kategori', nama_kategori)
7      aplikasi = []
8      for j in range(5):
9          nama_aplikasi = input('Nama aplikasi: ')
10         aplikasi.append(nama_aplikasi)
11         data_aplikasi[nama_kategori] = set(aplikasi)
12
13 print('\nDaftar aplikasi per kategori:')
14 for kategori, aplikasi in data_aplikasi.items():
15     print(kategori, ':', aplikasi)
16
17 daftar_set = list(data_aplikasi.values())
18
19 semua = daftar_set[0]
20 for s in daftar_set[1:]:
21     semua = semua.intersection(s)
22 print('\nAplikasi yang muncul di semua kategori:', semua)
23
24 print('\nAplikasi yang hanya muncul di satu kategori:')
25 for kategori, aplikasi in data_aplikasi.items():
26     gabungan_lain = set()
27     for kategori_lain, aplikasi_lain in data_aplikasi.items():
28         if kategori_lain != kategori:
29             gabungan_lain = gabungan_lain.union(aplikasi_lain)
30     hanya_satu = aplikasi.difference(gabungan_lain)
31     print(kategori, ':', hanya_satu)
32
33 if n > 2:
34     print('\nAplikasi yang muncul tepat di dua kategori:')
35     kategori_list = list(data_aplikasi.keys())
36     tepat_dua = set()
37     for i in range(len(kategori_list)):
38         for j in range(i+1, len(kategori_list)):
39             irisan = data_aplikasi[kategori_list[i]].intersection(data_aplikasi[kategori_list[j]])
40             # Pastikan tidak muncul di kategori lain
41             for k in range(len(kategori_list)):
42                 if k != i and k != j:
43                     irisan = irisan.difference(data_aplikasi[kategori_list[k]])
44             tepat_dua = tepat_dua.union(irisan)
45     print(tepat_dua)
```

Gambar 14.10: Input program dari soal 14.1. Sumber Python Vscode


```

Masukkan jumlah kategori: 2
Masukkan nama kategori: Finance
Masukkan 5 nama aplikasi di kategori Finance
Nama aplikasi: RTI Saham
Nama aplikasi: Mirae
Nama aplikasi: IPOT
Nama aplikasi: Poems
Nama aplikasi: Calculator
Masukkan nama kategori: Utilities
Masukkan 5 nama aplikasi di kategori Utilities
Nama aplikasi: Photos
Nama aplikasi: Weather
Nama aplikasi: Calculator
Nama aplikasi: Camera
Nama aplikasi: Notes

Daftar aplikasi per kategori:
Finance : {'IPOT', 'RTI Saham', 'Calculator', 'Poems', 'Mirae'}
Utilities : {'Calculator', 'Notes', 'Camera', 'Weather', 'Photos'}

Aplikasi yang muncul di semua kategori: {'Calculator'}

Aplikasi yang hanya muncul di satu kategori:
Finance : {'Poems', 'RTI Saham', 'Mirae', 'IPOT'}
Utilities : {'Camera', 'Weather', 'Notes', 'Photos'}

```

Gambar 14.11: Output program dari soal 14.1. Sumber Python Vscode

Program ini digunakan untuk mendata aplikasi berdasarkan kategori, lalu mencari aplikasi yang muncul di semua kategori, hanya ada di satu kategori, dan yang muncul tepat di dua kategori. Di awal program, pengguna diminta memasukkan jumlah kategori menggunakan `input()`, misalnya 2 kategori yaitu Finance dan Utilities. Setelah itu program menyiapkan dictionary kosong bernama `data_aplikasi` untuk menyimpan data kategori beserta daftar aplikasinya. Selanjutnya program melakukan perulangan sesuai jumlah kategori yang dimasukkan. Pengguna diminta memasukkan nama kategori, lalu memasukkan 5 nama aplikasi untuk kategori tersebut. Nama-nama aplikasi sementara disimpan dalam list aplikasi, kemudian diubah menjadi `set()` saat dimasukkan ke dictionary.

Penggunaan Set bertujuan agar data aplikasi menjadi unik dan tidak ada duplikasi. Setelah semua data dimasukkan, program menampilkan daftar aplikasi dari setiap kategori menggunakan perulangan `for`. Output yang muncul berupa nama kategori beserta isi Set aplikasinya. Berikutnya

program mencari aplikasi yang muncul di semua kategori menggunakan operasi intersection. Awalnya variabel semua diisi dengan Set pertama, lalu dibandingkan dengan Set kategori lain satu per satu menggunakan intersection(). Hasil akhirnya adalah aplikasi yang ada di seluruh kategori.

Program juga mencari aplikasi yang hanya muncul di satu kategori. Caranya dengan menggabungkan aplikasi dari kategori lain menggunakan union(), lalu mencari selisihnya memakai difference(). Hasilnya adalah aplikasi yang hanya ada di kategori tersebut dan tidak muncul di kategori lain. Bagian terakhir digunakan untuk mencari aplikasi yang muncul tepat di dua kategori. Program membandingkan setiap kategori menggunakan intersection(), lalu menghapus aplikasi yang ternyata muncul juga di kategori lain memakai difference(). Semua hasil tersebut kemudian digabungkan ke dalam Set tepat_dua.

Contoh sederhana input:

- Kategori Finance -> RTI Saham, Mirae, IPOT, Poems, Calculator.
- Kategori Utilities -> Photos, Weather, Calculator, Camera, Notes.

Contoh output:

- Aplikasi yang muncul di semua kategori → {Calculator}
- Aplikasi yang hanya muncul di satu kategori → aplikasi yang tidak ada di kategori lain
- Aplikasi yang muncul tepat di dua kategori → misalnya {Calculator}

Kesimpulannya, program ini memanfaatkan tipe data Set dan operasi himpunan seperti union, intersection, dan difference untuk mengolah data aplikasi berdasarkan kategori dengan lebih mudah dan efisien.

LATIHAN 14.2

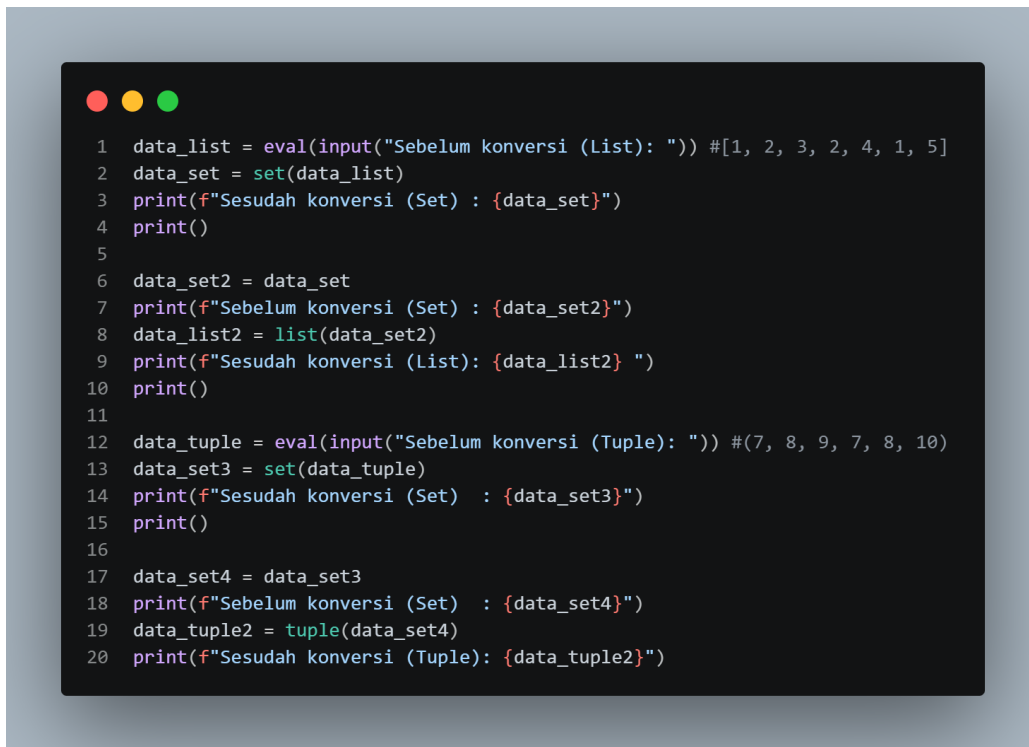
Buatlah sebuah program yang mendemonstrasikan konversi dari:

- List menjadi Set
- Set menjadi List

- Tuple menjadi Set
- Set menjadi Tuple

Tampilkan isi data sebelum dan sesudah konversi.

JAWABAN:

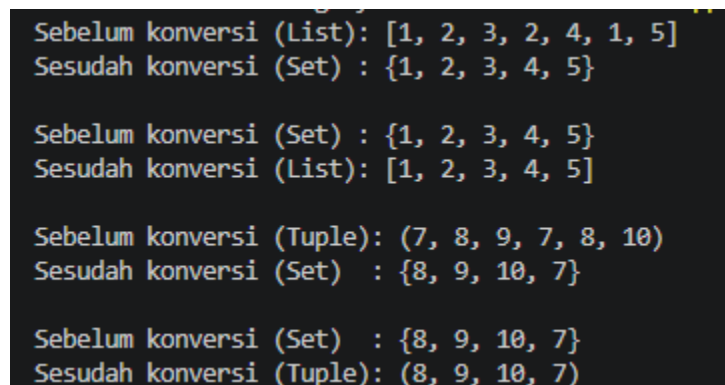


```

1 data_list = eval(input("Sebelum konversi (List): ")) #[1, 2, 3, 2, 4, 1, 5]
2 data_set = set(data_list)
3 print(f"Sesudah konversi (Set) : {data_set}")
4 print()
5
6 data_set2 = data_set
7 print(f"Sebelum konversi (Set) : {data_set2}")
8 data_list2 = list(data_set2)
9 print(f"Sesudah konversi (List): {data_list2} ")
10 print()
11
12 data_tuple = eval(input("Sebelum konversi (Tuple): ")) #(7, 8, 9, 7, 8, 10)
13 data_set3 = set(data_tuple)
14 print(f"Sesudah konversi (Set) : {data_set3}")
15 print()
16
17 data_set4 = data_set3
18 print(f"Sebelum konversi (Set) : {data_set4}")
19 data_tuple2 = tuple(data_set4)
20 print(f"Sesudah konversi (Tuple): {data_tuple2}")

```

Gambar 14.12: Input program dari soal 14.2. Sumber Python Vscode



```

Sebelum konversi (List): [1, 2, 3, 2, 4, 1, 5]
Sesudah konversi (Set) : {1, 2, 3, 4, 5}

Sebelum konversi (Set) : {1, 2, 3, 4, 5}
Sesudah konversi (List): [1, 2, 3, 4, 5]

Sebelum konversi (Tuple): (7, 8, 9, 7, 8, 10)
Sesudah konversi (Set) : {8, 9, 10, 7}

Sebelum konversi (Set) : {8, 9, 10, 7}
Sesudah konversi (Tuple): (8, 9, 10, 7)

```

Gambar 14.13: Output program dari soal 14.2. Sumber Python Vscode

Program ini digunakan untuk mengubah atau mengonversi tipe data antara List, Set, dan Tuple. Selain itu, program juga menunjukkan bahwa tipe data Set hanya menyimpan data unik sehingga data yang duplikat akan otomatis dihapus.

Pada bagian pertama, pengguna diminta memasukkan data bertipe List, misalnya [1, 2, 3, 2, 4, 1, 5]. Data tersebut kemudian diubah menjadi Set menggunakan fungsi `set()`. Saat dikonversi, angka yang duplikat seperti 1 dan 2 akan hilang karena Set hanya menyimpan anggota unik. Output yang muncul misalnya menjadi {1, 2, 3, 4, 5}. Setelah itu, data Set tadi dikonversi kembali menjadi List menggunakan fungsi `list()`. Hasilnya data kembali berbentuk List, tetapi isi datanya tetap unik karena sebelumnya sudah diproses oleh Set.

Pada bagian berikutnya, pengguna memasukkan data bertipe Tuple, misalnya (7, 8, 9, 7, 8, 10). Data Tuple tersebut diubah menjadi Set menggunakan `set()`, sehingga data yang sama akan otomatis dihapus dan hasilnya menjadi data unik saja. Terakhir, data Set tersebut dikonversi kembali menjadi Tuple memakai fungsi `tuple()`. Hasil output akhirnya berupa Tuple yang hanya berisi data unik tanpa duplikasi.

LATIHAN 14.3

Buatlah sebuah program yang dapat membaca dua file teks dan menampilkan semua kata-kata yang muncul pada kedua file tersebut. Beberapa hal yang perlu anda perhatikan:

- Nama file adalah input user. Tampilkan pesan error jika file tidak ditemukan/tidak bisa dibaca.
- Semua kata dikonversi dulu menjadi lowercase.
- Sertakan contoh file teks yang anda pakai saat mengumpulkan laporan

JAWABAN:

(di page berikutnya :v)

```

1  nama_file1 = input("Masukkan nama file pertama: ")
2  nama_file2 = input("Masukkan nama file kedua: ")
3
4  try:
5      file1 = open(nama_file1, 'r')
6      file2 = open(nama_file2, 'r')
7      isi_file1 = file1.read().lower().split()
8      isi_file2 = file2.read().lower().split()
9      file1.close()
10     file2.close()
11     set_file1 = set(isi_file1)
12     set_file2 = set(isi_file2)
13 except:
14     print("Error: File", nama_file1, "tidak ditemukan/tidak bisa dibaca!")
15     exit()
16     print("Error: File", nama_file2, "tidak ditemukan/tidak bisa dibaca!")
17     exit()
18
19 # Tampilkan kata-kata di setiap file
20 print("\nKata-kata di", nama_file1, ":", set_file1)
21 print("Kata-kata di", nama_file2, ":", set_file2)
22
23 # Tampilkan kata yang muncul di KEDUA file (intersection)
24 kata_sama = set_file1.intersection(set_file2)
25 print("\nKata-kata yang muncul di kedua file:", kata_sama)

```

Gambar 14.14: Input program dari soal 14.3. Sumber Python Vscode

```

Masukkan nama file pertama: file1.txt
Masukkan nama file kedua: file2.txt

Kata-kata di file1.txt : {'favorit', 'makan', 'nasi', 'pakai', 'adalah', 'suka', 'saya', 'makanan', 'goreng', 'mie', 'dan'}
Kata-kata di file2.txt : {'minum', 'putih', 'favorit', 'juga', 'air', 'adalah', 'suka', 'saya', 'susu', 'minuman', 'matcha', 'dan'}

Kata-kata yang muncul di kedua file: {'favorit', 'adalah', 'suka', 'saya', 'dan'}
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/python.exe "d:/GitHub/Training Dylan/Soal03.py
Masukkan nama file pertama: file1.txt
Masukkan nama file kedua: file3.txt
Error: File file1.txt tidak ditemukan/tidak bisa dibaca!

```

Gambar 14.15: Output program dari soal 14.3. Sumber Python Vscode

Program ini digunakan untuk membaca dua file teks lalu mencari kata-kata yang muncul di kedua file menggunakan operasi intersection pada Set. Di awal program, pengguna diminta memasukkan nama file pertama dan file kedua menggunakan input(). Setelah itu, program membuka kedua file dengan mode baca ('r'), lalu isi file dibaca menggunakan read(). Semua huruf

diubah menjadi huruf kecil memakai `lower()` agar kata seperti “Python” dan “python” dianggap sama. Kemudian teks dipisahkan menjadi kata-kata menggunakan `split()`.

Data kata dari masing-masing file kemudian diubah menjadi Set menggunakan `set()`. Tujuannya agar kata yang sama atau duplikat di dalam file hanya muncul satu kali. Program juga memakai `try-except` untuk menangani error jika file tidak ditemukan atau tidak bisa dibaca. Selanjutnya program menampilkan semua kata unik dari file pertama dan file kedua. Setelah itu dilakukan operasi `intersection()` untuk mencari kata-kata yang muncul di kedua file sekaligus. Hasilnya disimpan di variabel `kata_sama` lalu ditampilkan ke output.

Contoh sederhana:

- File 1 -> “Saya suka makan Nasi Goreng dan Mie Goreng pakai nasi. Nasi Goreng adalah makanan favorit saya.”
- File 2 -> “Saya suka minum Matcha dan Susu juga Air putih. Matcha adalah minuman favorit saya.”

Output:

- Kata di file 1 → {'adalah', 'pakai', 'nasi', 'favorit', 'makan', 'saya', 'dan', 'suka', 'mie', 'goreng', 'makanan'}
- Kata di file 2 → {'adalah', 'matcha', 'putih', 'minuman', 'favorit', 'saya', 'air', 'dan', 'suka', 'susu', 'minum', 'juga'}
- Kata yang sama → {'adalah', 'favorit', 'saya', 'dan', 'suka'}